Introducere

Datele

Structuri de date tabelare

- Fisiere de tip spreadsheet - în care datele pot fi de tipuri diferite (sir de caractere, numeric, dată sau text), dar fiecare coloana va contine date de acelasi tip:
  - coloanele = atributele (caracteristicile)
  - liniile = exemplele
  - Acestea includ majoritatea tipurilor de date stocate în mod obișnuit în baze de date relaționale.
- Fișiere text delimitate de \tab sau virgulă, etc.

Tablouri multidimensionale(matrici)

Librarii Python

- NumPy
- Pandas
- Matplotlib
- Scikit-learn

NumPy

- Pachet de programe pentru calcule matematice si numerice in Python.
- Obiect *ndarray* de tip multidimensional array – rapid si eficient
- *NumPy arrays* – cele mai eficiente structuri de date pentru stocarea si manipularea datelor
- Functii predefinite pentru calculul cu matrici si functii matematice (ca si in matlab)
- Tools pentru citire si scriere structuri de date sub forma de arrays in fisiere
- Operatii cu vectori si matrici, valori proprii, vectori proprii, algebra liniara, generare numere pseudoaleatoare
- https://numpy.org/doc/1.20/user/index.html

Pandas

- Libraria Python pentru analiza datelor
- Pachet de programe care ofera structuri de date de nivel inalt si functii care fac lucrul cu date structurate sau tabelare rapid, usor si expresiv.

Obiectele principale

- Series
- DataFrame

Matplotlib

- Cea mai populara librarie Python pentru realizarea de grafice si vizualizari de date in 2D.

scikit-learn

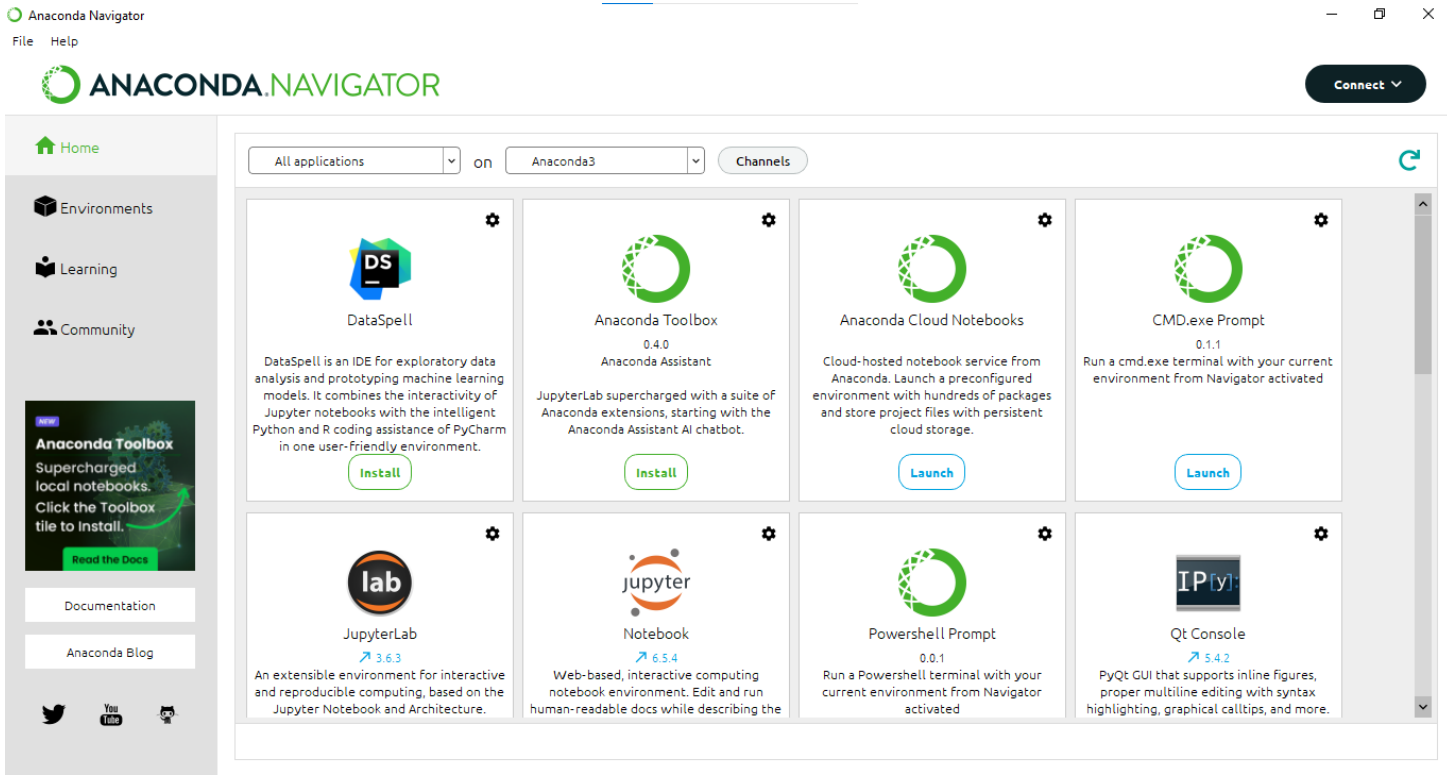
- Principalul tool pentru ML pentru programatorii in Python
- Include module pentru operatii precum:
  - **Preprocesare:** Feature extraction, normalization
  - **Clasificare**: SVM, nearest neighbors, random forest, logistic regression, etc.
  - **Regresie**: Lasso, ridge regression, etc.
  - **Clusterizare**: *k*-means, spectral clustering, etc.
  - **Reducerea dimensionalitatii**: PCA, feature selection, matrix factorization, etc.
  - **Selectarea modelului**: Grid search, cross-validation, metrics

Anaconda

- IPython and Jupyter
- https://www.anaconda.com/
- una din cele mai populare platforme pentru data science
- free, open-source environment
- Contine peste 7500 colectii de pachete/librarii scrise in Python/R

Instalare Anaconda si Setup

- free Anaconda distribution [Anaconda | Anaconda Distribution]
- To get started on Windows, download the Anaconda installer.
- Anaconda Navigator
- Jupiter Notebook
- Ultima versiune 2.5.3 (15.03.2024)

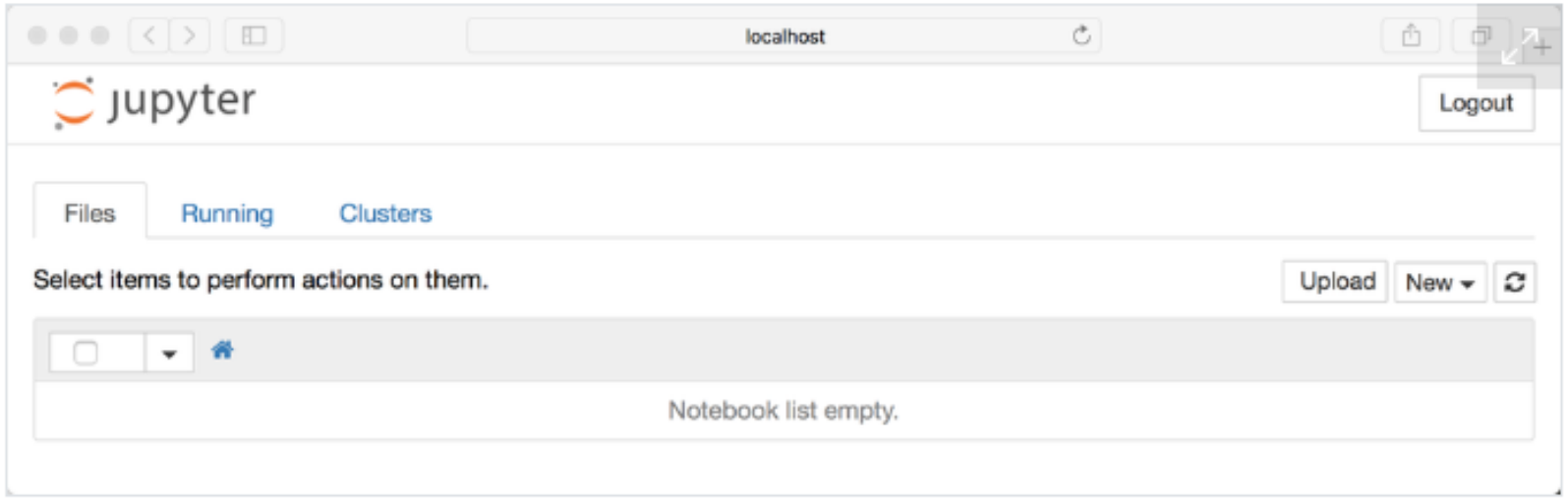


Jupyter Notebook

- Launch – Jupyter Notebook
- Server Jupyter
  http://localhost:8888/tree

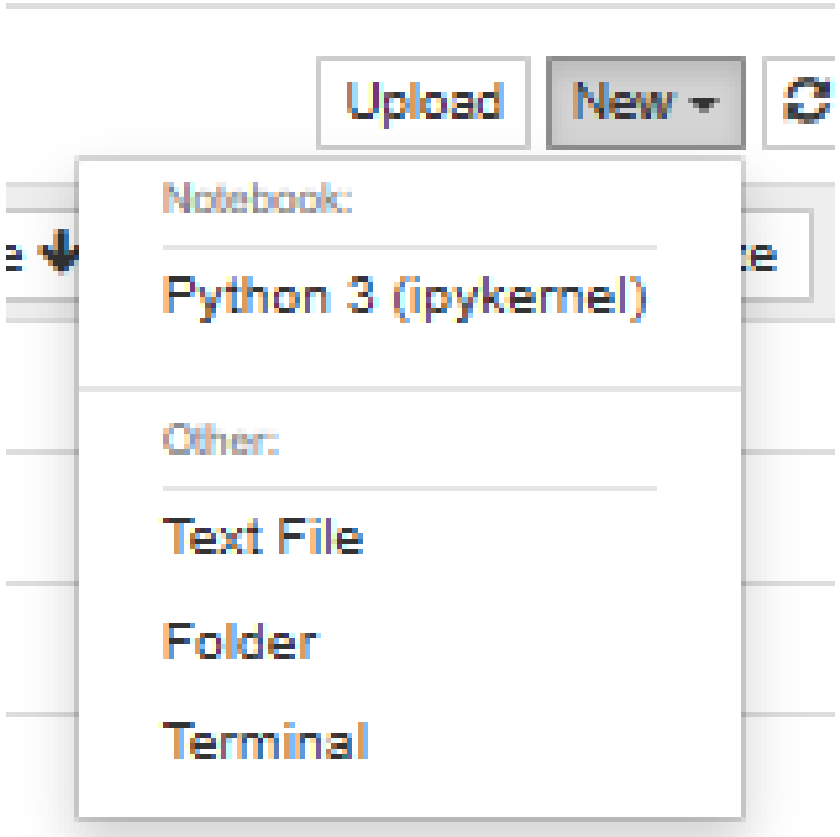
  Se va deschide in browser

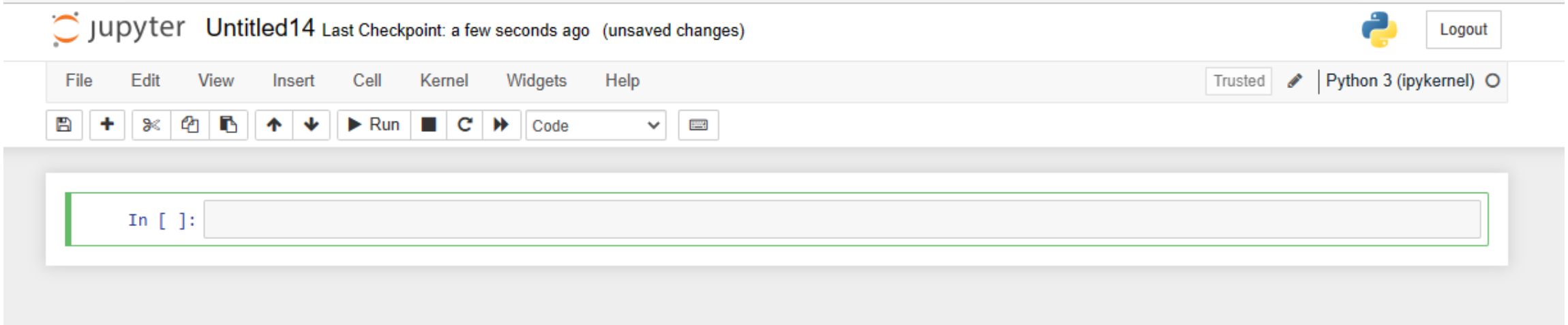
Your browser should now look something like this:

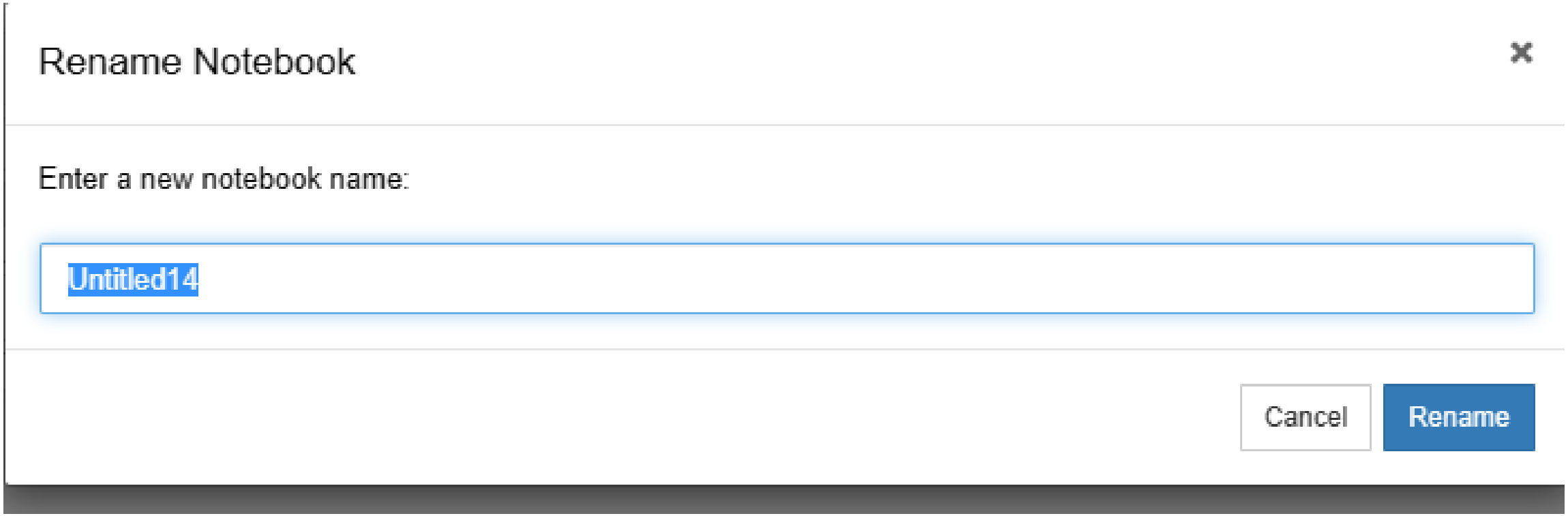


Jupyter Notebook

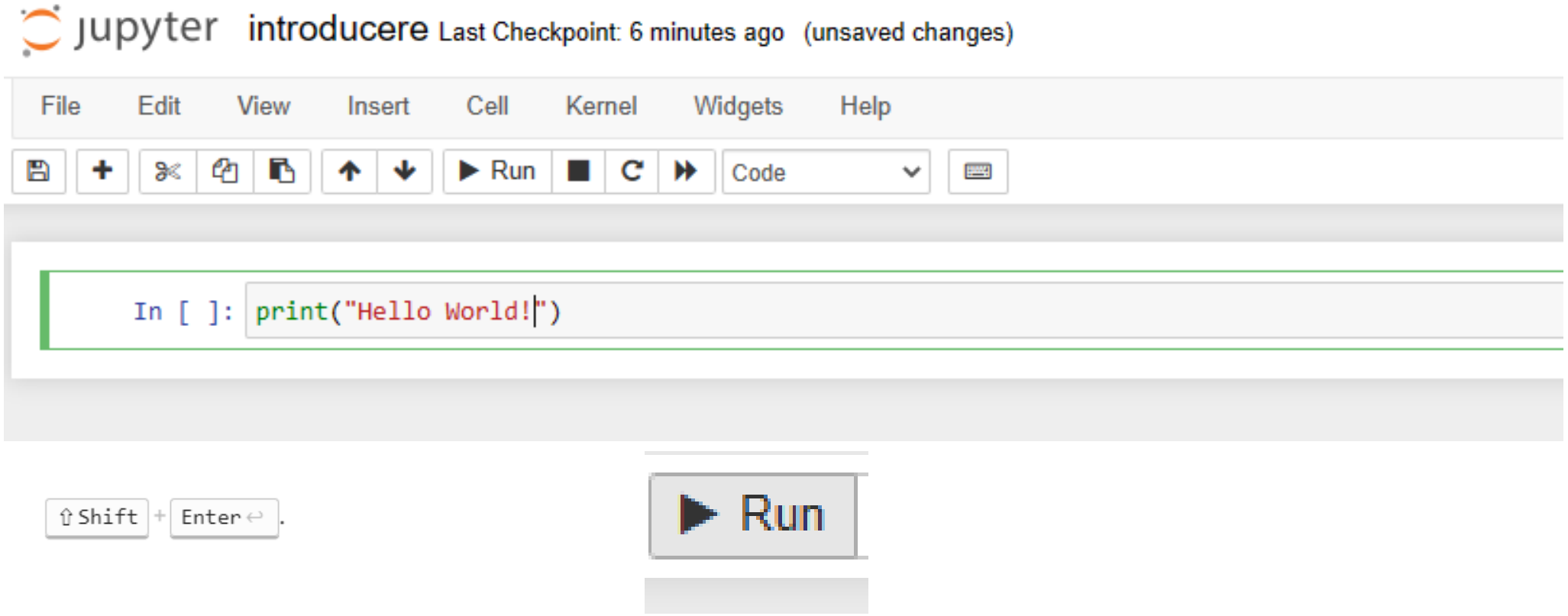
Pentru a deschide un nou notebook.




jupyter
Untitled14
Last Checkpoint: a few seconds ago
(unsaved changes)
Logout
File
Edit
View
Insert
Cell
Kernel
Widgets
Help
Trusted
Python 3 (ipykernel)
Run
Code
In [ ]:

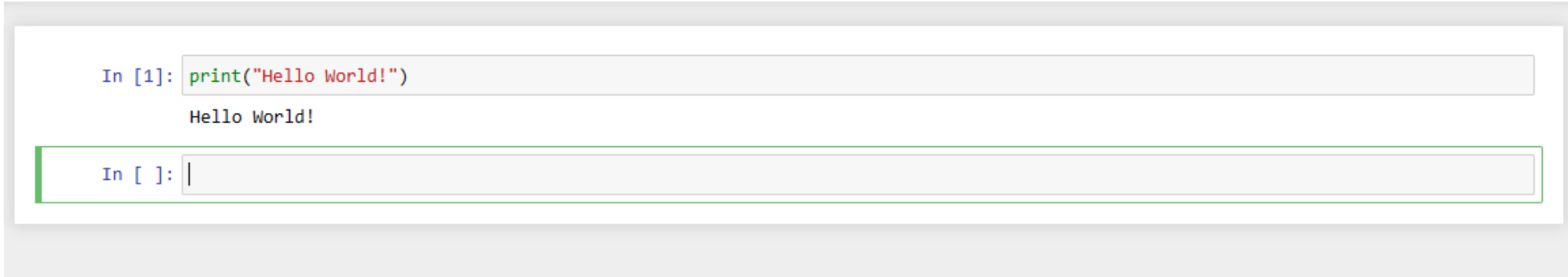


http://localhost:8889/notebooks/introducere.ipynb


jupyter introducere Last Checkpoint: 6 minutes ago (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [ ]: print("Hello World!")

Shift + Enter .

Run


In [1]: print("Hello World!")
Hello World!
In [ ]:

- Dupa ca am rulat o celula, langa apare

```
In [1]: print("Hello World!")

        Hello World!
```

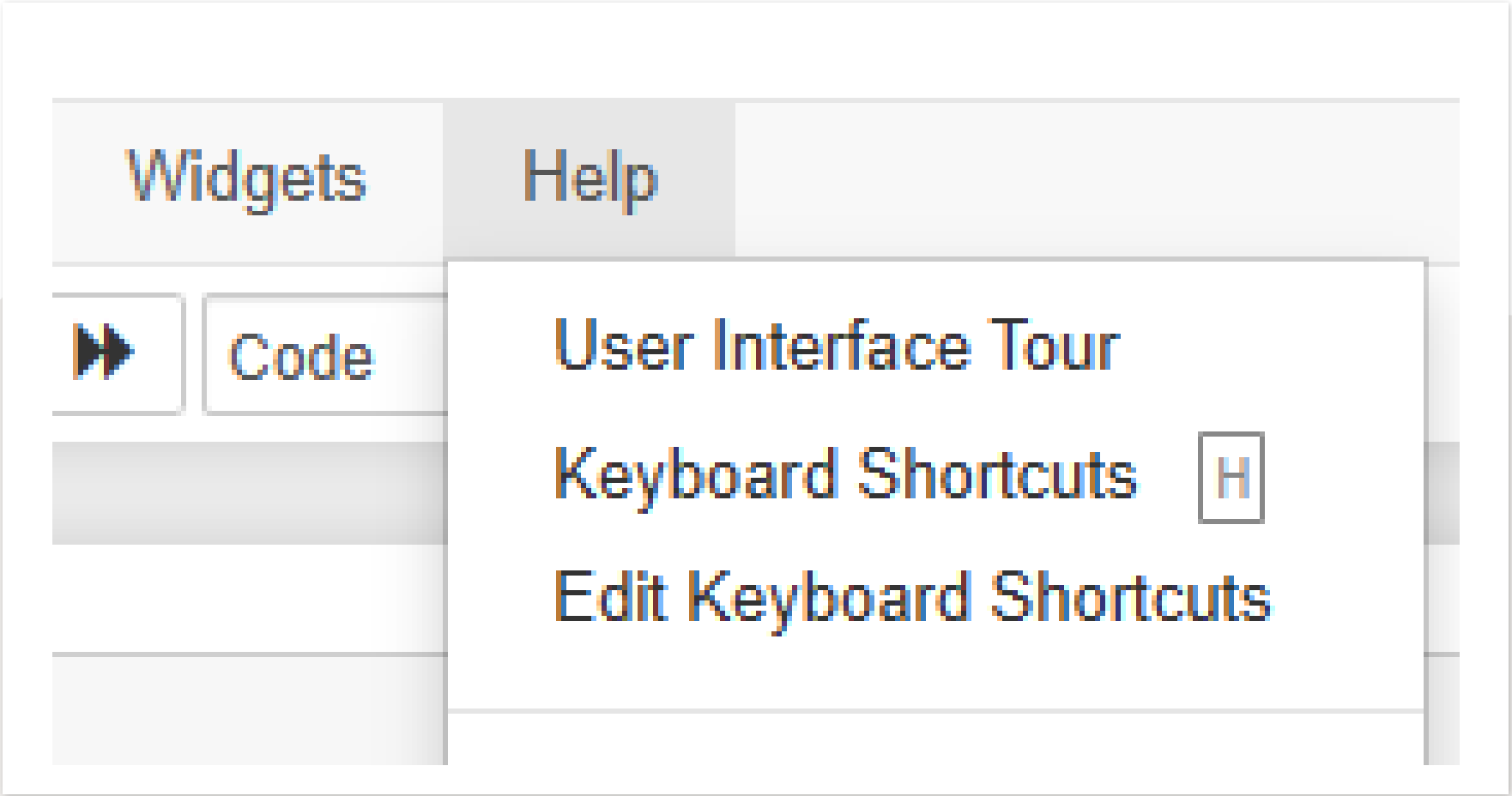
- Daca celula nu a fost rulata apare doar [ ]
- La fiecare rulare, intre [ ] se va scrie urmatorul numar. Nu conteaza ordinea in notebook, ci conteaza ordinea in care au fost executate instructiunile (ordinea numerotarii).

```
In [1]: print("Hello World!")

        Hello World!

In [3]: print("Azi" + " e sambata!")

        Azi e sambata!
```

- Daca rularea unei celule dureaza, apare [*].

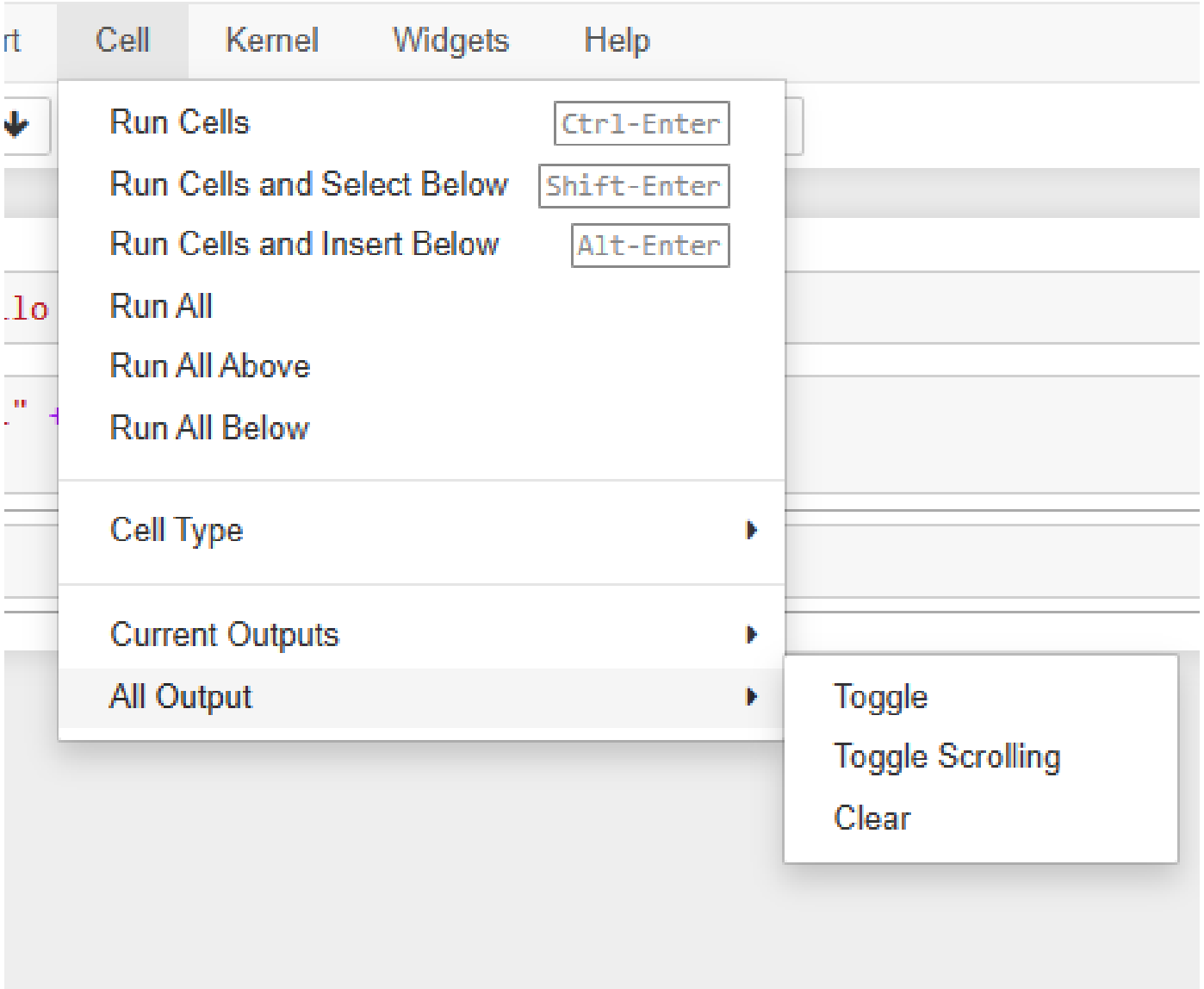
- Nu conteaza ordinea in notebook.

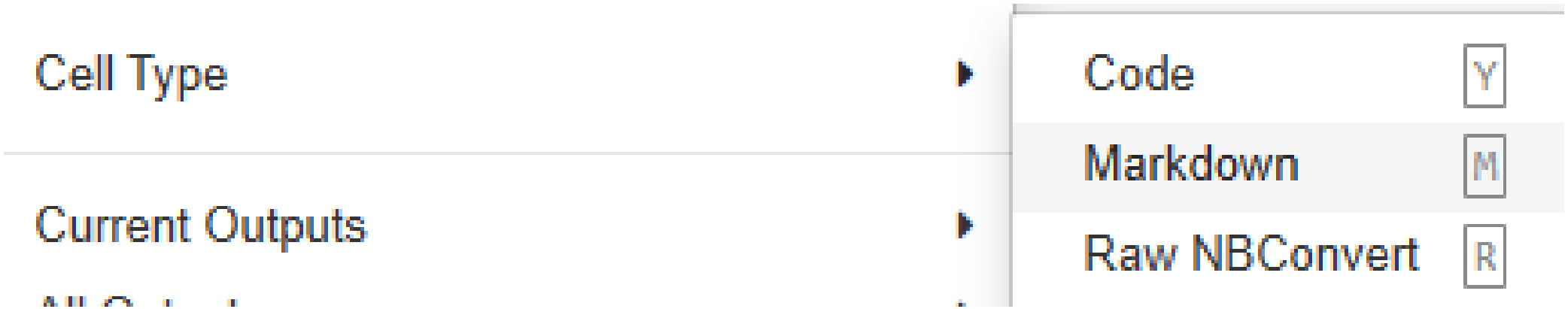
```
In [4]: print("Hello World!")

        Hello World!

In [3]: print("Azi" + " e sambata!")

        Azi e sambata!

In [ ]:
```



User interface tour

- Cell - > Toggle ascunde output-ul unei celule.
- Cell - > Clear sterge output-ul unei celule.
- Stergem all output -> clear; sau Toggle





Tipuri de celule (Cell Type)

- Jupyter Notebook acceptă Markdown, care este un limbaj de marcare construit pe HTML.

- In Edit Mode

```
Daca vrem sa scriem cu italic _un text_ atunci punem *underscore sau  steluta*.

Daca vrem sa scriem cu bold __un text__ atunci punem doua underscore sau **doua stelute**.

Pentru titlu folosim hastag.
```

```
# Titlu
## Subtitlu
```

Daca vrem sa scriem cu italic *un text* atunci punem *underscore sau steluta*.

Daca vrem sa scriem cu bold **un text** atunci punem doua underscore sau **doua stelute**.

Pentru titlu folosim hastag.

Titlu

Subtitlu

```
* unu
* doi
* trei
```

```
+ unu
+ doi
```

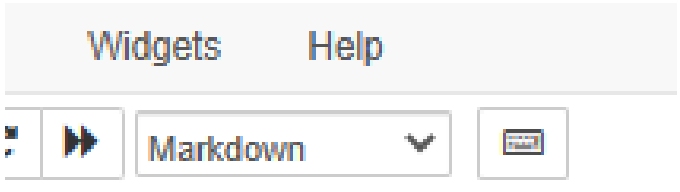
- unu
- doi
- trei

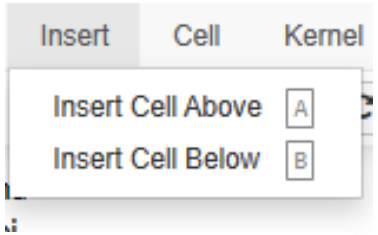
- unu
- doi

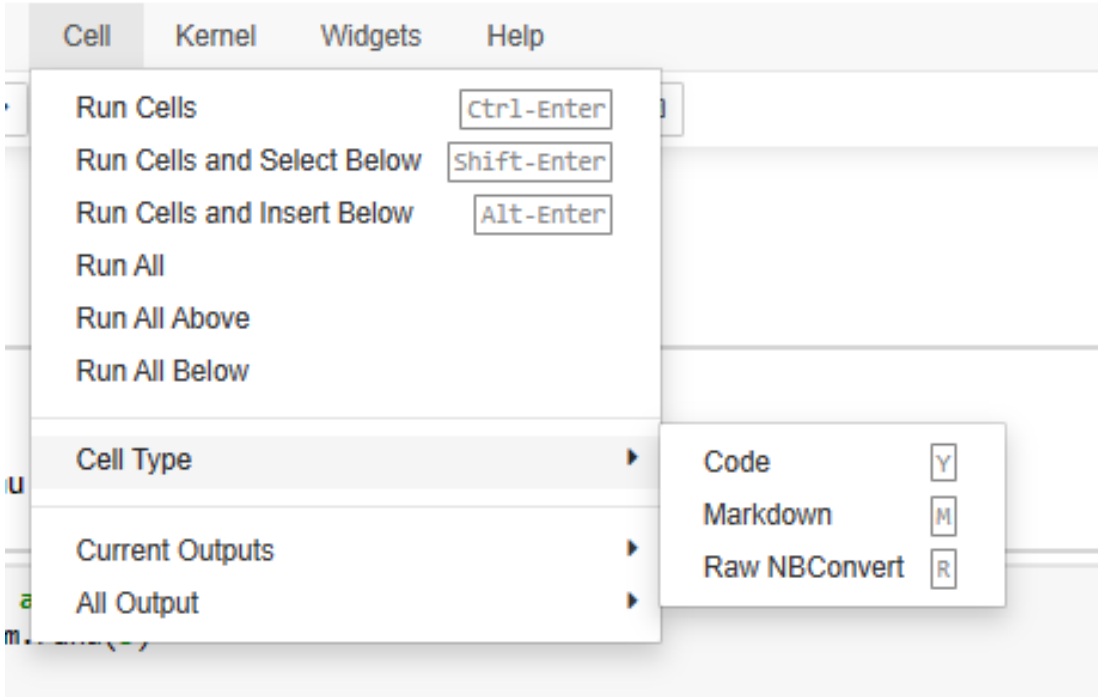
```
+ unu
+ doi
  + doi.unu
```

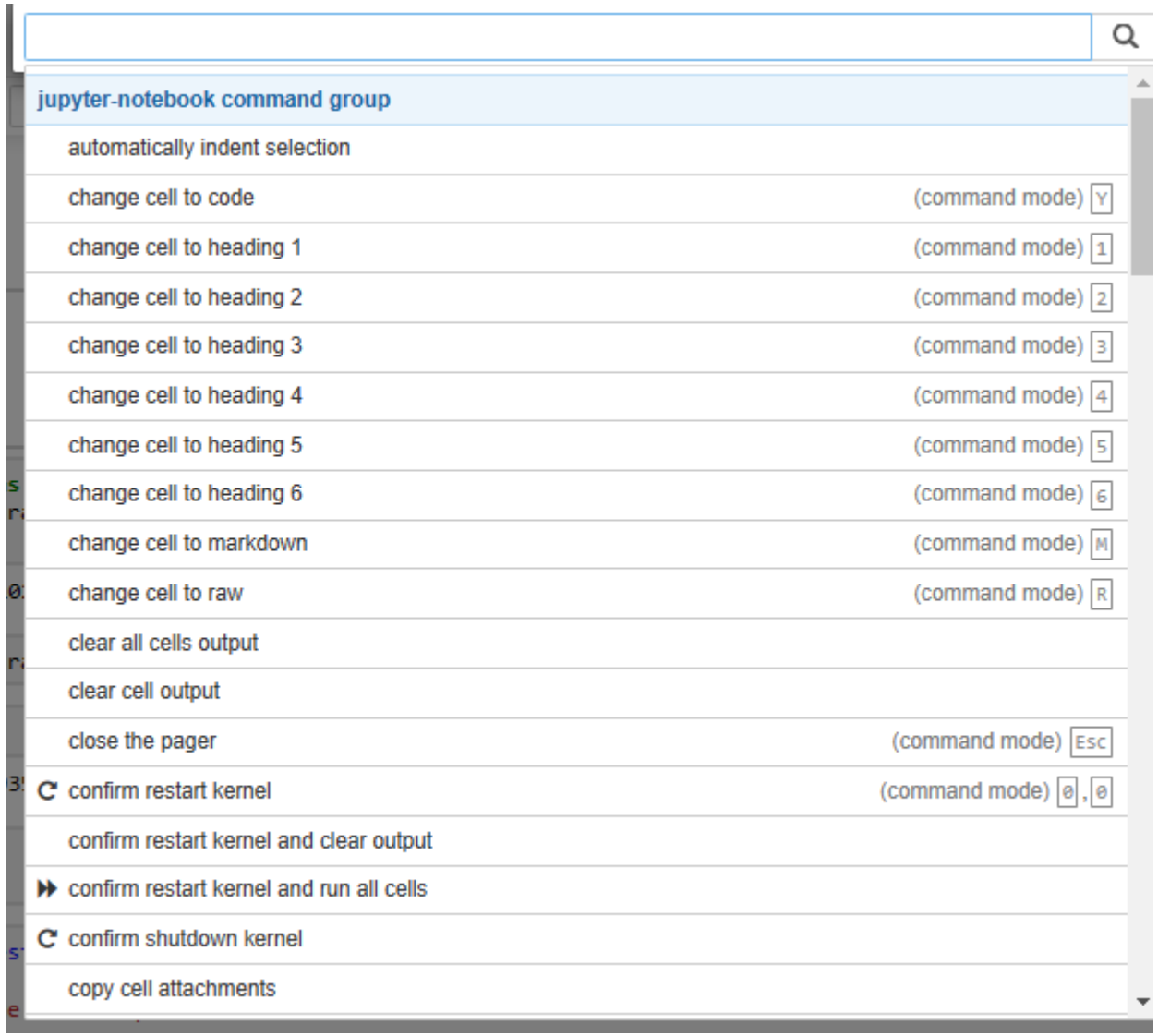
- unu
- doi
    - doi.unu

Keyboard Shortcuts



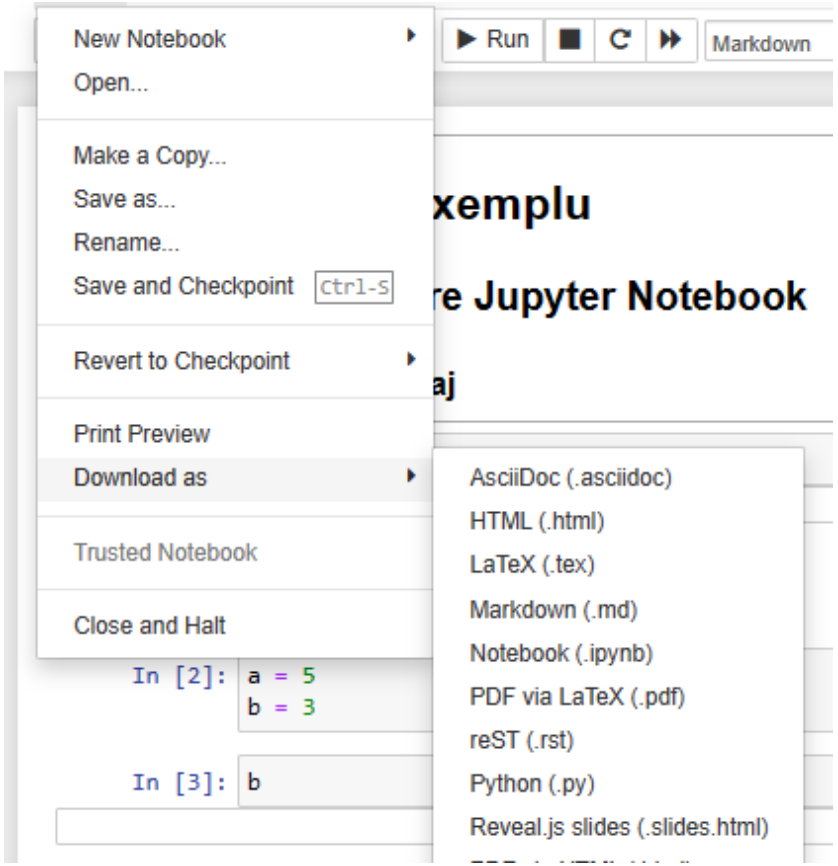






- Fisierul creat are extensia *.ipynb* si contine tot, inclusiv rezultatele obtinute la rularea notebook-ului.

- Poate fi salvat sub forma de mai multe tipuri de fisiere.



Import - Conventii

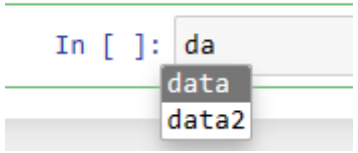
- Pentru importul celor mai populare librarii, se folosesc urmatoarele conventii:
  - **import numpy as np**
  - **import matplotlib.pyplot as plt (sau from matplotlib import pyplot as plt)**
  - **import pandas as pd**
  - **import seaborn as sns**
  - **import statsmodels as sm**

In [8]:
```
import numpy as np
a = np.random.rand(3)
a
```

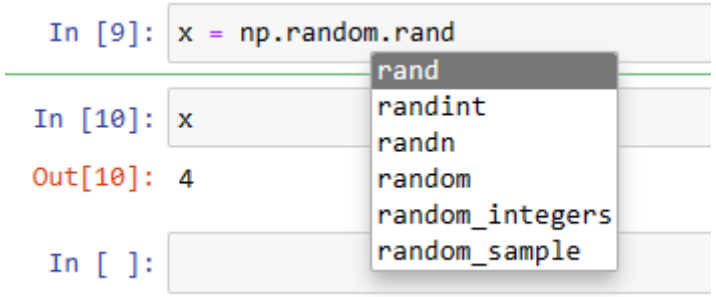
Out[8]:
```
array([0.67781725, 0.11085343, 0.25611314])
```

Completare Tab

- Atunci cand suntem in Cell -> code daca apasam tab numele de variabile, de functii se completeaza:



- La fel daca apasam tab dupa punct, la apelarea de attribute sau metode pentru un obiect:



Introspectie

- Daca folosim semnul intrebarii (?) înainte sau după **o variabilă** se vor va afișa unele informații generale despre obiect.

```
In [8]: import numpy as np
        a = np.random.rand(3)
        a
Out[8]: array([0.67781725, 0.11085343, 0.25611314])
```

```
In [9]: a?
```

```
Type:        ndarray
String form: [0.67781725 0.11085343 0.25611314]
Length:      3
File:        c:\programdata\anaconda3\lib\site-packages\numpy\__init__.py
Docstring:
ndarray(shape, dtype=float, buffer=None, offset=0,
        strides=None, order=None)

An array object represents a multidimensional, homogeneous array
of fixed-size items.  An associated data-type object describes the
format of each element in the array (its byte-order, how many bytes it
occupies in memory, whether it is an integer, a floating point number,
or something else, etc.)

Arrays should be constructed using `array`, `zeros` or `empty` (refer
to the See Also section below).  The parameters given here refer to
a low-level method (`ndarray(...)`) for instantiating an array.
```

```
For more information, refer to the `numpy` module and examine the
methods and attributes of an array.

Parameters
----------
(for the __new__ method; see Notes below)

shape : tuple of ints
    Shape of created array.
dtype : data-type, optional
    Any object that can be interpreted as a numpy data type.
buffer : object exposing buffer interface, optional
    Used to fill the array with data.
offset : int, optional
    Offset of array data in buffer.
strides : tuple of ints, optional
```

```
    Strides of data in memory.
order : {'C', 'F'}, optional
    Row-major (C-style) or column-major (Fortran-style) order.

Attributes
----------
T : ndarray
    Transpose of the array.
data : buffer
    The array's elements, in memory.
dtype : dtype object
    Describes the format of the elements in the array.
flags : dict
    Dictionary containing information related to memory use, e.g.,
    'C_CONTIGUOUS', 'OWNDATA', 'WRITEABLE', etc.
flat : numpy.flatiter object
    Flattened version of the array as an iterator.  The iterator
    allows assignments, e.g., ``x.flat = 3`` (See `ndarray.flat` for
    assignment examples; TODO).
imag : ndarray
    Imaginary part of the array.
real : ndarray
    Real part of the array.
size : int
    Number of elements in the array.
itemsize : int
    The memory use of each array element in bytes.
nbytes : int
    The total number of bytes required to store the array data,
    i.e., ``itemsize * size``.
ndim : int
    The array's number of dimensions.
shape : tuple of ints
    Shape of the array.
strides : tuple of ints
    The step-size required to move from one element to the next in
    memory. For example, a contiguous ``(3, 4)`` array of type
    ``int16`` in C-order has strides ``(8, 2)``.  This implies that
    to move from element to element in memory requires jumps of 2 bytes.
    To move from row-to-row, one needs to jump 8 bytes at a time
    (``2 * 4``).
ctypes : ctypes object
    Class containing properties of the array needed for interaction
    with ctypes.
base : ndarray
    If the array is a view into another array, that array is its `base`
    (unless that array is also a view).  The `base` array is where the
    array data is actually stored.

See Also
--------
array : Construct an array.
zeros : Create an array, each element of which is zero.
empty : Create an array, but leave its allocated memory unchanged (i.e.,
        it contains "garbage").
dtype : Create a data-type.
numpy.typing.NDArray : A :term:`generic <generic type>` version
                       of ndarray.

Notes
-----
There are two modes of creating an array using ``__new__``:

1. If `buffer` is None, then only `shape`, `dtype`, and `order`
   are used.
2. If `buffer` is an object exposing the buffer interface, then
   all keywords are interpreted.

No ``__init__`` method is needed because the array is fully initialized
after the ``__new__`` method.

Examples
--------
These examples illustrate the low-level `ndarray` constructor.  Refer
to the `See Also` section above for easier ways of constructing an
ndarray.

First mode, `buffer` is None:

>>> np.ndarray(shape=(2,2), dtype=float, order='F')
array([[0.0e+000, 0.0e+000], # random
       [     nan, 2.5e-323]])

Second mode:

>>> np.ndarray((2,), buffer=np.array([1,2,3]),
...            offset=np.int_().itemsize,
...            dtype=int) # offset = 1*itemsize, i.e. skip first element
array([2, 3])
```

Introspectie

- Daca obiectul este o **functie sau o metoda**, daca a fost inclus in definitia functiei si comentariul initial (docstring), la aplicarea ?, va fi afisat si acest comentariu.

```
In [12]: def functie_test_comentarii():
             """
             Acesta este un test pentru comentariu si ?
             """
             print("testam....")

In [13]: functie_test_comentarii()
         testam....

In [14]: functie_test_comentarii?

In [ ]:
```

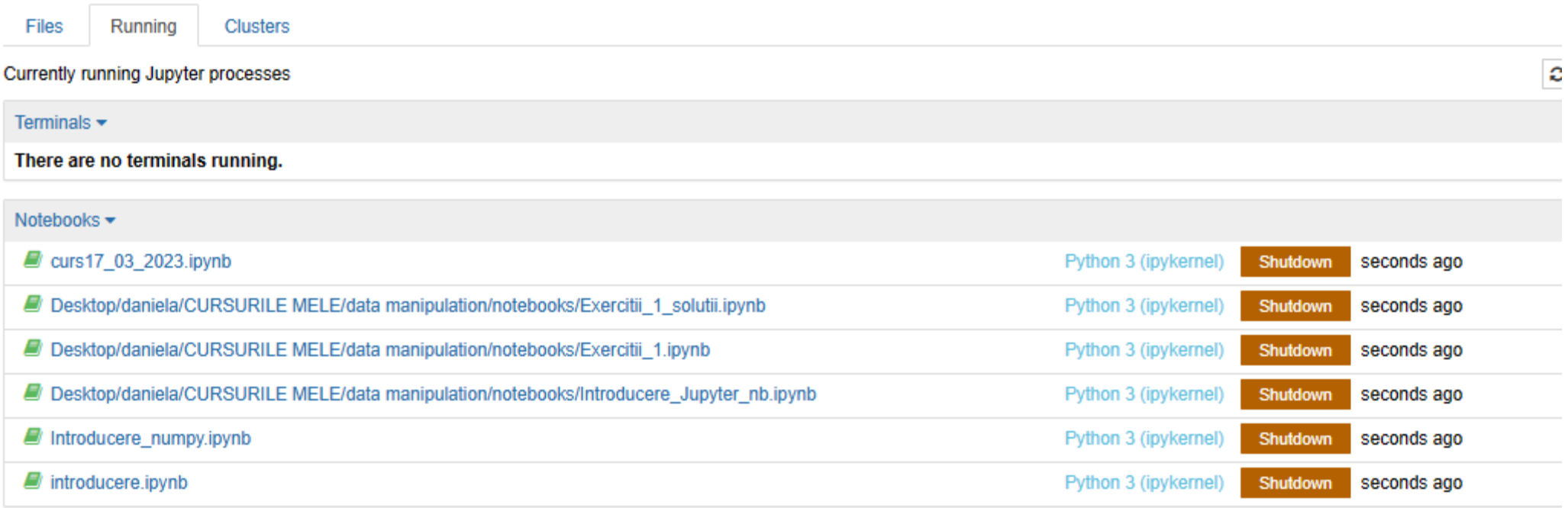
```
Signature: functie_test_comentarii()
Docstring: Acesta este un test pentru comentariu si ?
File:      c:\users\daniela\appdata\local\temp\ipykernel_20788\3565122970.py
Type:      function
```

Daca folosim ?? va apare si corpul functiei

```
In [15]: functie_test_comentarii??
```

```
Signature: functie_test_comentarii()
Source:
def functie_test_comentarii():
    """
    Acesta este un test pentru comentariu si ?
    """
    print("testam....")
File:      c:\users\daniela\appdata\local\temp\ipykernel_20788\3565122970.py
Type:      function
```

- Notebook-urile rămân în funcțiune până când le închideți în mod explicit; închiderea paginii notebook-ului nu este suficientă.
-



Exercitii _1

- Exercitii_1 - Jupyter Notebook